Index of Experiments

1. Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation. 2
2. Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python. 5
3. Perform sampling and reconstruction of continuous signal using MATLAB/ Python. . 7
4. To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python. 10
5. Perform the PAM modulation and demodulation using MATLAB/ Python. 12
6. Perform the DPCM modulation and demodulation using MATLAB/ Python. 14
7. To develop a program and simulate the Pulse Amplitude Modulation (PAM) signal and analyze its waveform. 16
8. To design a program and simulate the Amplitude Shift Keying (ASK) modulation technique. 19
9. To construct a program and simulate the Phase Shift Keying (PSK) modulation technique. 22
10. To create a program and simulate the Frequency Shift Keying (FSK) modulation technique. 25
11. To implement a program and simulate the Unipolar Non-Return-to-Zero (NRZ) signaling technique. 28
12. To model a program and simulate the Polar Non-Return-to-Zero (NRZ) signaling technique. 30
13. To write a program and simulate the Unipolar Return-to-Zero (RZ) signaling technique. 32
14. To simulate a program for the Bipolar Return-to-Zero (RZ) signaling technique. 34
15. To develop a program and simulate the Manchester (Split-Phase) coding technique. . 36

Experiment No: 1

Title

Perform the QPSK modulation and demodulation using MATLAB/ Python. Display the signal and its constellation.

Objectives

- To understand the working principle of QPSK modulation.
- To implement QPSK modulation and demodulation using Python.
- To analyze the waveform and constellation diagram.

Theory

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique where two bits are transmitted simultaneously by varying the phase of a carrier signal. This increases bandwidth efficiency compared to binary modulation schemes.

In QPSK, binary data is grouped into pairs and each pair is mapped into a symbol represented by in-phase (I) and quadrature (Q) components. These components modulate cosine and sine carrier signals respectively.

The QPSK signal is expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t) \tag{1}$$

where $I(t)$ and $Q(t)$ represent the in-phase and quadrature components and f_c is the carrier frequency.

The symbol mapping is defined as:

$$(0,0) \to (1,1), \quad (0,1) \to (-1,1), \quad (1,1) \to (-1,-1), \quad (1,0) \to (1,-1) \tag{2}$$

At the receiver, demodulation is performed by detecting the sign of the I and Q components to recover the original binary data.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randint(0, 2, 20)
pairs = data.reshape(-1, 2)

map_dict = {(0,0):(1,1), (0,1):(-1,1), (1,1):(-1,-1), (1,0):(1,-1)}
symbols = np.array([map_dict[tuple(p)] for p in pairs])

I, Q = symbols[:,0], symbols[:,1]

t = np.linspace(0, 1, 100)
```

```
fc = 5
signal = np.concatenate([
    I[i]*np.cos(2*np.pi*fc*t) + Q[i]*np.sin(2*np.pi*fc*t)
    for i in range(len(I))
])

demod = []
for i,q in zip(I,Q):
    if i>0 and q>0: demod += [0,0]
    elif i<0 and q>0: demod += [0,1]
    elif i<0 and q<0: demod += [1,1]
    else: demod += [1,0]

demod = np.array(demod)

plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.step(range(len(data)), data, where='mid')
plt.title("Original Binary Signal")
plt.xlabel("Bit Index")
plt.ylabel("Amplitude")
plt.ylim(-0.5, 1.5)
plt.grid()

plt.subplot(2,2,2)
plt.plot(signal)
plt.title("QPSK Modulated Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(2,2,3)
plt.step(range(len(demod)), demod, where='mid')
plt.title("Demodulated Signal")
plt.xlabel("Bit Index")
plt.ylabel("Amplitude")
plt.ylim(-0.5, 1.5)
plt.grid()

plt.subplot(2,2,4)
plt.scatter(I, Q)
plt.title("QPSK Constellation")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.tight_layout()

print("Original Data:", data)
print("Demodulated Data:", demod)
plt.show()
```

Output

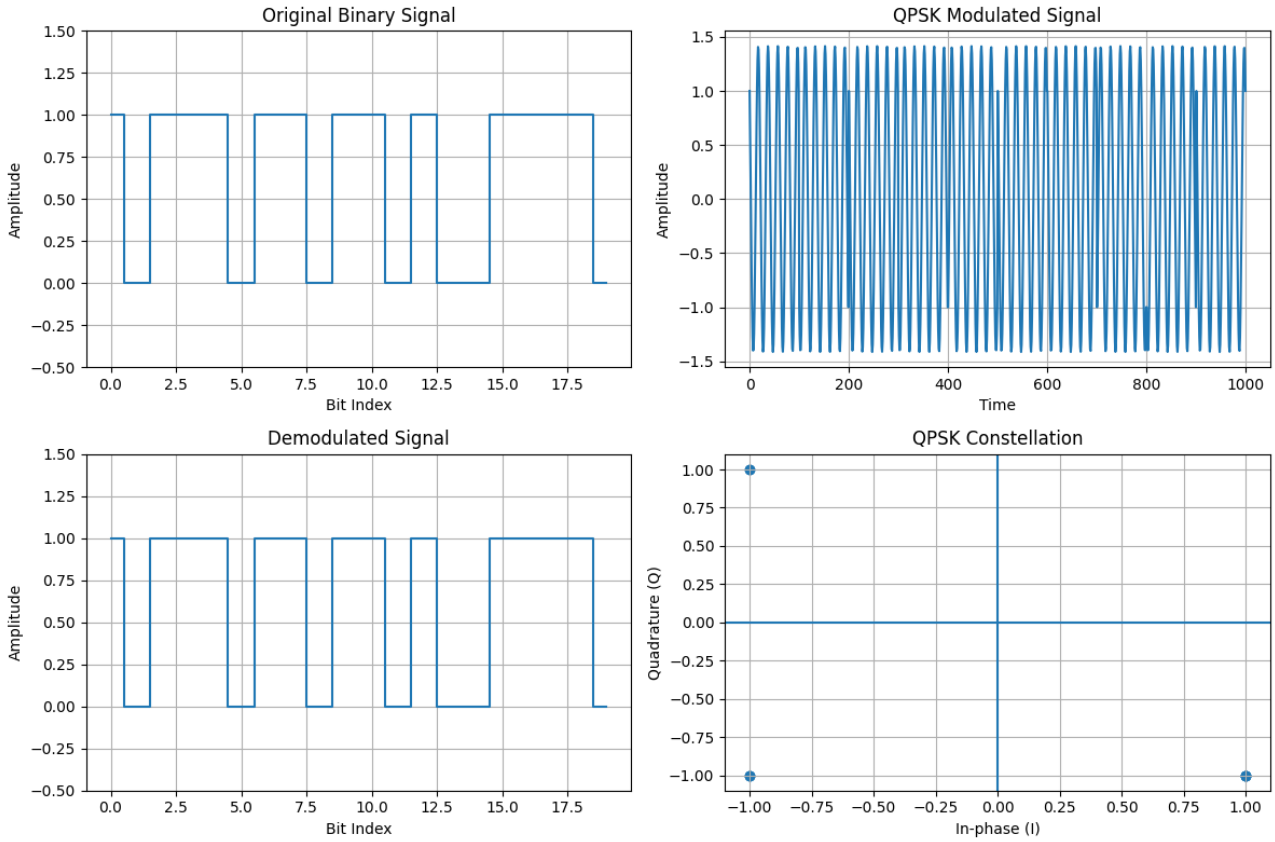

Figure 1: QPSK Modulated Signal, Demodulated Signal and Constellation Diagram

Results and Discussions

The QPSK modulation converts binary data into phase-shifted signals efficiently. The demodulated output closely matches the original input sequence. The constellation diagram shows four clearly separated symbol points. This indicates proper mapping and accurate detection of symbols. Thus, the system demonstrates correct modulation and demodulation.

Experiment No: 2

Title

Generate 16-QAM modulation and obtain the QAM constellation using MATLAB/ Python.

Objectives

- To understand the working principle of 16-QAM modulation.
- To generate modulated QAM signals using Python.
- To analyze the waveform and constellation diagram.

Theory

Quadrature Amplitude Modulation (QAM) is a digital modulation technique that combines both amplitude and phase variations of a carrier signal. In 16-QAM, each symbol represents 4 bits, which increases the data transmission rate.

The QAM signal is expressed as:

$$s(t) = I(t)\cos(2\pi f_c t) + Q(t)\sin(2\pi f_c t) \tag{1}$$

where $I(t)$ and $Q(t)$ are the in-phase and quadrature components.

In 16-QAM, the amplitude levels are defined as:

$$I, Q \in \{-3, -1, +1, +3\} \tag{2}$$

Gray coding is used to map bits into symbols to minimize the probability of bit errors.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 200
bits = np.random.randint(0, 2, N*4)
bit_groups = bits.reshape((N, 4))

mapping = {
    (0,0): -3,
    (0,1): -1,
    (1,1):  1,
    (1,0):  3
}

I = np.array([mapping[tuple(b[:2])] for b in bit_groups])
Q = np.array([mapping[tuple(b[2:])] for b in bit_groups])

qam_signal = I + 1j * Q
t = np.arange(N)
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(t, np.real(qam_signal), label='In-phase (I)')
plt.plot(t, np.imag(qam_signal), label='Quadrature (Q)')
plt.title("16-QAM Modulated Signal")
plt.xlabel("Symbol Index")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()

plt.subplot(1,2,2)
plt.scatter(np.real(qam_signal), np.imag(qam_signal))
plt.title("16-QAM Constellation Diagram")
plt.xlabel("In-phase (I)")
plt.ylabel("Quadrature (Q)")
plt.xticks([-3, -1, 1, 3])
plt.yticks([-3, -1, 1, 3])
plt.grid()
plt.axis('equal')

plt.tight_layout()
plt.show()
```

Output

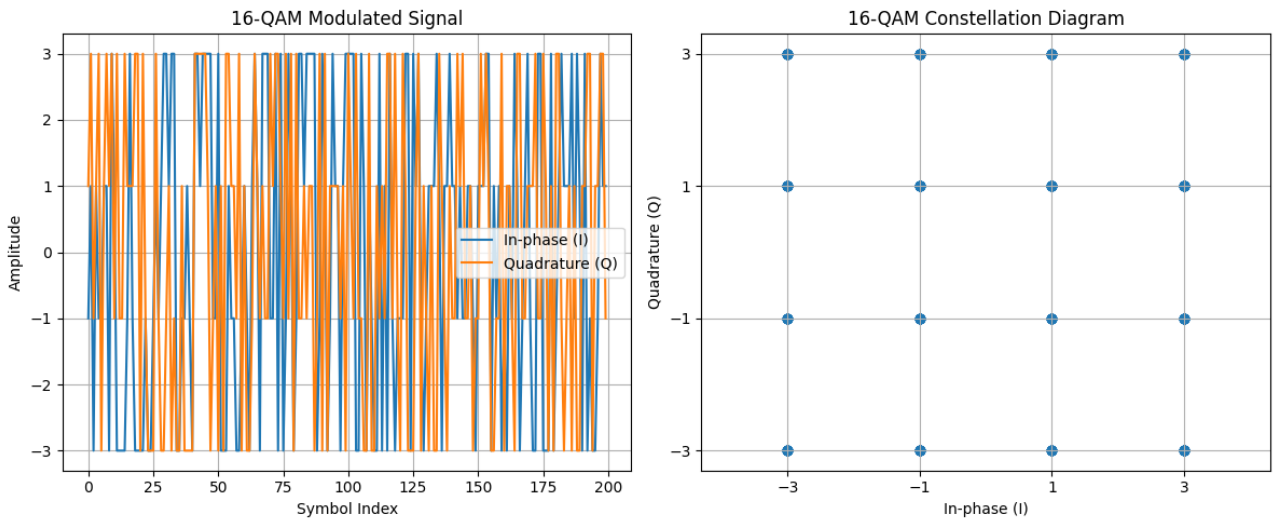

Figure 2: 16-QAM Modulated Signal and Constellation Diagram

Results and Discussions

The 16-QAM modulation generates signals using multiple amplitude levels. Both in-phase and quadrature components vary with symbol index. The constellation diagram shows 16 distinct symbol points. Each point represents a unique 4-bit combination. Thus, the system efficiently increases data transmission rate.

Experiment No: 3

Title

Perform sampling and reconstruction of continuous signal using MATLAB/ Python.

Objectives

- To understand the concept of signal sampling.
- To perform sampling of a continuous-time signal.
- To reconstruct the original signal from sampled data.

Theory

Sampling is the process of converting a continuous-time signal into a discrete-time signal by taking measurements at regular time intervals. According to the Nyquist sampling theorem, the sampling frequency must be at least twice the highest frequency of the signal to avoid aliasing.

A continuous-time sinusoidal signal is represented as:

$$x(t) = \sin(2\pi f t) \tag{1}$$

The sampled signal is obtained as:

$$x[n] = x(nT_s) \tag{2}$$

where $T_s = \frac{1}{f_s}$ is the sampling interval and f_s is the sampling frequency.

The reconstruction of the signal is performed using interpolation techniques to approximate the original continuous signal.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d

t = np.linspace(0, 1, 1000)
f = 5
x = np.sin(2 * np.pi * f * t)

fs = 20
ts = np.arange(0, 1, 1/fs)
xs = np.sin(2 * np.pi * f * ts)

interp_func = interp1d(ts, xs, kind='linear')
t_new = np.linspace(ts.min(), ts.max(), 1000)
x_recon = interp_func(t_new)

plt.figure(figsize=(12,8))
```

```
plt.subplot(3,1,1)
plt.plot(t, x)
plt.title("Original Continuous Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(3,1,2)
plt.stem(ts, xs, basefmt=" ")
plt.title("Sampled Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(3,1,3)
plt.plot(t, x, label="Original Signal")
plt.plot(t_new, x_recon, '--', label="Reconstructed Signal")
plt.scatter(ts, xs)
plt.title("Signal Reconstruction")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

Output

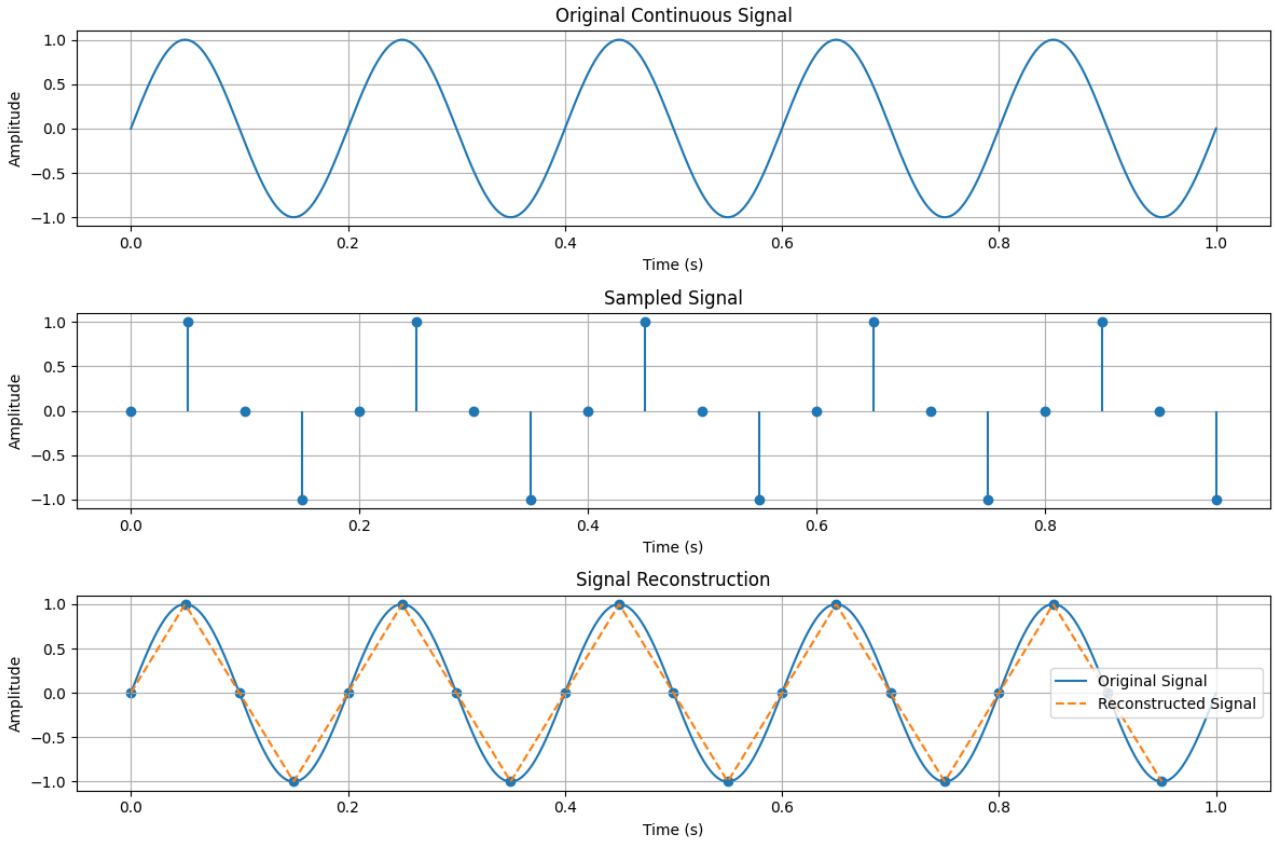

Figure 3: Sampling and Reconstruction of Continuous Signal

Results and Discussions

The continuous signal is successfully sampled at a fixed sampling frequency. The sampled signal represents discrete-time values of the original signal. The reconstructed signal closely follows the original waveform. Minor differences occur due to interpolation method used. Thus, proper sampling enables accurate signal reconstruction.

Experiment No: 4

Title

To generate the waveform of Quadrature Phase Shift Keying signal using MATLAB/ Python.

Objectives

- To understand the generation of QPSK waveform.
- To analyze in-phase and quadrature components.
- To observe the final QPSK modulated signal.

Theory

Quadrature Phase Shift Keying (QPSK) is a modulation technique in which two bits are transmitted per symbol by changing the phase of the carrier signal. The signal is formed using two orthogonal carriers: cosine (in-phase) and sine (quadrature).

The QPSK signal is given by:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t) \tag{1}$$

where $I(t)$ and $Q(t)$ represent the in-phase and quadrature components.

The binary input is grouped into pairs and mapped into corresponding I and Q values using Gray coding. These components are then multiplied with carrier signals to generate the final QPSK waveform.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
num_bits = 20; fc = 5; samples_per_bit = 100

bits = np.random.randint(0, 2, num_bits)
bit_pairs = bits.reshape(-1, 2)
mapping = {
    (0,0):(1,1), (0,1):(-1,1),
    (1,1):(-1,-1), (1,0):(1,-1)
}
I, Q = [], []
for pair in bit_pairs:
    i, q = mapping[tuple(pair)]
    I += [i]*samples_per_bit
    Q += [q]*samples_per_bit
I, Q = np.array(I), np.array(Q)
t = np.arange(len(I)) / samples_per_bit

carrier_I = np.cos(2*np.pi*fc*t)
```

```
carrier_Q = np.sin(2*np.pi*fc*t)
qpsk_signal = I*carrier_I - Q*carrier_Q

plt.figure(figsize=(10,6))
plt.subplot(3,1,1)
plt.plot(t, I)
plt.title("In-phase (I)")

plt.subplot(3,1,2)
plt.plot(t, Q)
plt.title("Quadrature (Q)")

plt.subplot(3,1,3)
plt.plot(t, qpsk_signal)
plt.title("QPSK Signal")
plt.tight_layout()
plt.show()
```

Output

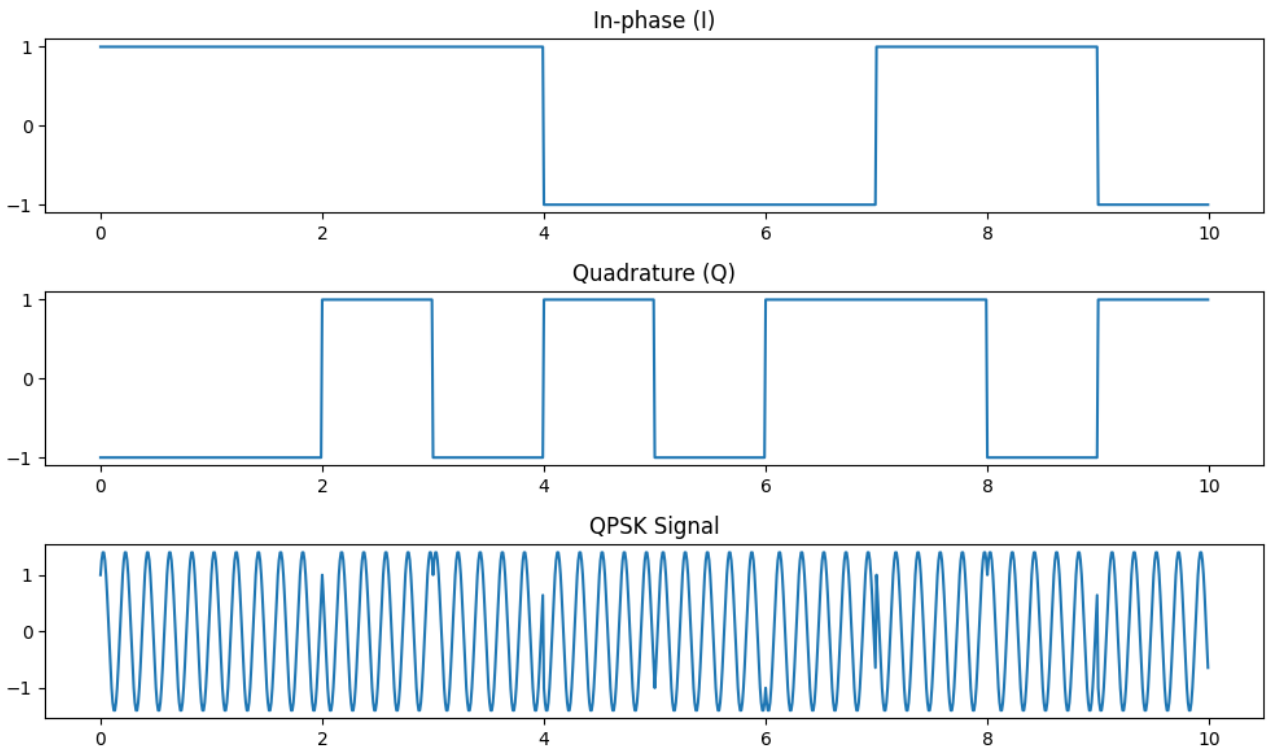

Figure 4: In-phase, Quadrature and QPSK Waveform

Results and Discussions

The in-phase and quadrature signals are generated correctly from bit pairs. These components vary according to the input binary data. The final QPSK signal is formed by combining both carriers. The waveform shows phase variations corresponding to symbols. Thus, the QPSK signal generation is successfully verified.

Experiment No: 5

Title

Perform the PAM modulation and demodulation using MATLAB/ Python.

Objectives

- To understand the concept of Pulse Amplitude Modulation (PAM).
- To implement PAM modulation and demodulation.
- To analyze the effect of noise and signal recovery.

Theory

Pulse Amplitude Modulation (PAM) is a technique where the amplitude of equally spaced pulses is varied according to the message signal. In digital PAM, binary data is mapped into discrete amplitude levels.

The PAM signal can be represented as:

$$s(t) = \sum_n a_n p(t - nT) \tag{1}$$

where a_n represents the amplitude corresponding to each symbol and $p(t)$ is the pulse shape.

For binary PAM, the mapping is defined as:

$$a_n = \begin{cases} +A, & \text{for bit 1} \\ -A, & \text{for bit 0} \end{cases} \tag{2}$$

At the receiver, demodulation is performed by sampling the received signal and applying a threshold decision rule to recover the original bits.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

num_bits = 20; samples_per_bit = 100; A = 5
bits = np.random.randint(0, 2, num_bits)
symbols = np.where(bits == 1, A, -A)
pam_signal = np.repeat(symbols, samples_per_bit)
t = np.arange(len(pam_signal)) / samples_per_bit

noise = np.random.normal(0, 1, len(pam_signal))
received_signal = pam_signal + noise
sample_idx = np.arange(samples_per_bit//2, len(received_signal),
   samples_per_bit)
samples = received_signal[sample_idx]

recovered_bits = (samples > 0).astype(int)
```

```
demod_symbols = np.where(recovered_bits == 1, A, -A)
demod_signal = np.repeat(demod_symbols, samples_per_bit)

plt.figure(figsize=(10,8))
plt.subplot(4,1,1)
plt.step(range(len(bits)), bits, where='post')
plt.title("Original Bits")
plt.subplot(4,1,2); plt.plot(t, pam_signal)
plt.title("PAM Modulated Signal")
plt.subplot(4,1,3)
plt.plot(t, received_signal)
plt.scatter(sample_idx/samples_per_bit, samples)
plt.title("Received Signal (with Sampling Points)")

plt.subplot(4,1,4)
plt.plot(t, demod_signal)
plt.title("Demodulated Signal")
plt.tight_layout()
plt.show()
print("Original bits:", bits)
print("Recovered bits:", recovered_bits)
```

Output

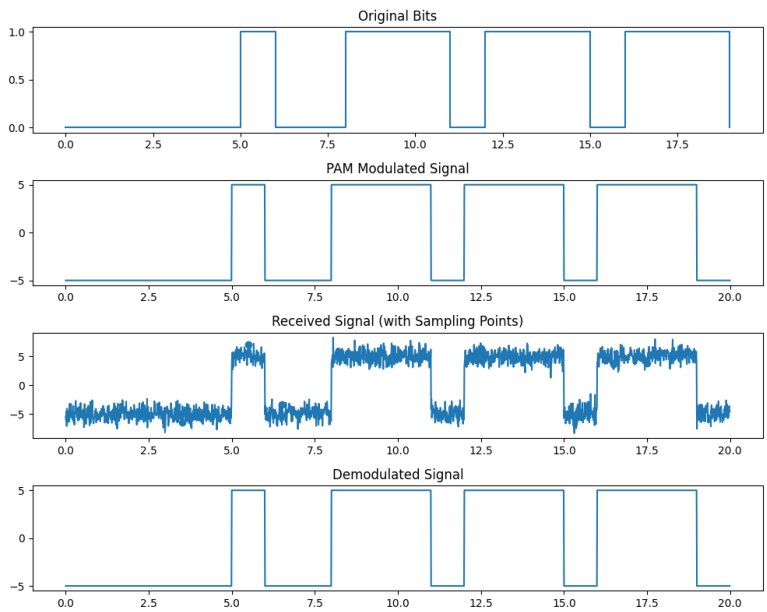

Figure 5: PAM Modulation and Demodulation with Noise

Results and Discussions

The PAM signal is generated by mapping binary data to amplitude levels. Noise affects the transmitted signal during propagation. Sampling is performed at optimal points for accurate detection. The recovered bits closely match the original transmitted bits. Thus, PAM modulation and demodulation are successfully implemented.

Experiment No: 6

Title

Perform the DPCM modulation and demodulation using MATLAB/ Python.

Objectives

- To understand the concept of Differential Pulse Code Modulation (DPCM).
- To implement DPCM encoding and decoding.
- To analyze prediction and quantization errors.

Theory

Differential Pulse Code Modulation (DPCM) is a signal encoding technique that transmits the difference between successive samples instead of the actual signal values. This reduces redundancy and improves efficiency.

The prediction error is given by:

$$e(n) = x(n) - \hat{x}(n) \tag{1}$$

where $x(n)$ is the original signal and $\hat{x}(n)$ is the predicted signal.

The quantized error is expressed as:

$$e_q(n) = Q[e(n)] \tag{2}$$

At the receiver, the reconstructed signal is obtained by:

$$x_r(n) = x_r(n-1) + e_q(n) \tag{3}$$

Thus, DPCM uses prediction and quantization to efficiently encode the signal.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 200)
x = np.sin(2 * np.pi * 5 * t)

step_size = 0.2
predicted = np.zeros_like(x)
error = np.zeros_like(x)
q_error = np.zeros_like(x)

for i in range(1, len(x)):
    predicted[i] = x[i-1]
    error[i] = x[i] - predicted[i]
    q_error[i] = step_size * np.round(error[i] / step_size)
```

```
reconstructed = np.zeros_like(x)
for i in range(1, len(x)):
    reconstructed[i] = reconstructed[i-1] + q_error[i]

plt.figure(figsize=(10,8))
plt.subplot(4,1,1)
plt.plot(t, x)
plt.title("Original Signal")
plt.subplot(4,1,2)
plt.plot(t, error)
plt.title("Prediction Error")

plt.subplot(4,1,3)
plt.plot(t, q_error)
plt.title("Quantized Error")
plt.subplot(4,1,4)
plt.plot(t, reconstructed)
plt.title("Reconstructed Signal")
plt.tight_layout()
plt.show()
```

Output

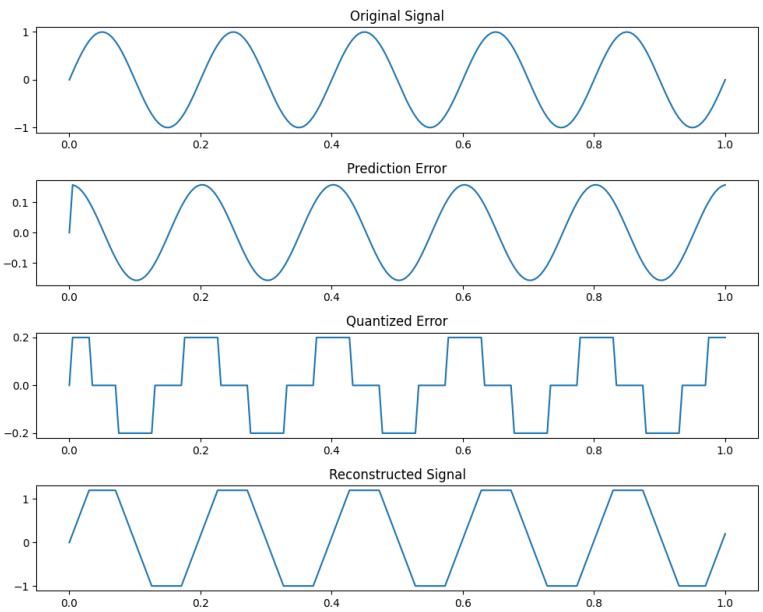

Figure 6: DPCM Modulation and Demodulation Process

Results and Discussions

The DPCM system encodes the difference between signal samples. Prediction reduces redundancy in the transmitted signal. Quantization introduces small errors in the signal. The reconstructed signal closely follows the original waveform. Thus, DPCM efficiently compresses and reconstructs the signal.

Experiment No: 7

Title

To develop a program and simulate the Pulse Amplitude Modulation (PAM) signal and analyze its waveform.

Objectives

- To understand the concept of Pulse Amplitude Modulation.
- To generate PAM signal using a message signal and sampling pulses.
- To analyze the waveform of PAM signal.

Theory

Pulse Amplitude Modulation (PAM) is a technique in which the amplitude of a pulse varies according to the instantaneous value of the message signal. In this experiment, PAM is generated by sampling a sinusoidal message signal using a periodic pulse train.

The message signal is defined as:

$$m(t) = \sin(2\pi f_m t) \tag{1}$$

A periodic square wave is used as a sampling pulse:

$$p(t) = \text{square}(2\pi f_c t) \tag{2}$$

The PAM signal is obtained by multiplying the message signal with the pulse train:

$$s(t) = m(t) \cdot p(t) \tag{3}$$

In discrete form, the PAM signal can be expressed as:

$$s[n] = \begin{cases} m[n], & \text{if sampling pulse is ON} \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

The duty cycle of the square wave determines the width of the sampling pulses, which controls how much of the signal is transmitted during each sampling period.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

fc, fm, fs, t, duty = 20, 2, 1000, 1, 20

n = np.arange(0, t, 1/fs)

s = (signal.square(2 * np.pi * fc * n, duty/100) + 1) / 2
```

```
m = np.sin(2 * np.pi * fm * (n - 1))

period = int(fs / fc)
on_samples = int(np.ceil(period * duty / 100))

pam = np.zeros_like(n)

for i in range(0, len(n), period):
    pam[i:i+on_samples] = m[i]

plt.figure(figsize=(8, 6))

plt.subplot(3,1,1)
plt.plot(n, s)
plt.ylim([-0.2, 1.2])
plt.title("Square Wave")

plt.subplot(3,1,2)
plt.plot(n, m)
plt.ylim([-1.2, 1.2])
plt.title("Message Signal")

plt.subplot(3,1,3)
plt.plot(n, pam)
plt.ylim([-1.2, 1.2])
plt.title("PAM Signal")

plt.tight_layout()
plt.show()
```

Output

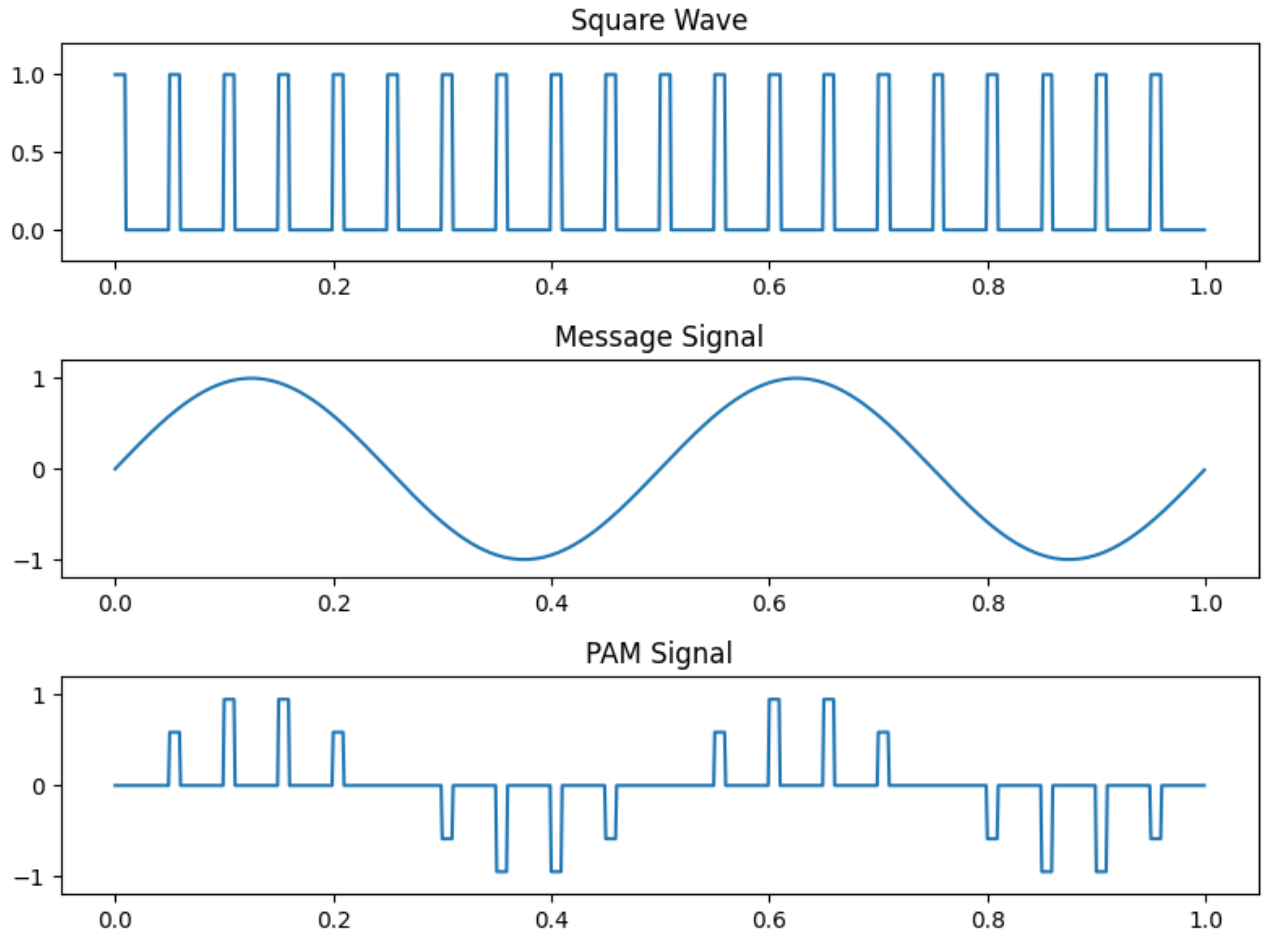

Figure 7: Square Wave, Message Signal and PAM Signal

Results and Discussions

The PAM signal is generated by sampling the message signal. The square wave acts as a sampling pulse train. The amplitude of pulses follows the message signal values. The PAM waveform clearly represents sampled signal behavior. Thus, Pulse Amplitude Modulation is successfully implemented.

Experiment No: 8

Title

To design a program and simulate the Amplitude Shift Keying (ASK) modulation technique.

Objectives

- To understand the principle of Amplitude Shift Keying modulation.
- To generate ASK modulated signal for binary data.
- To recover the original signal using demodulation.

Theory

Amplitude Shift Keying (ASK) is a digital modulation technique in which the amplitude of a carrier signal is varied according to the binary input data. It is widely used in digital communication systems for transmitting binary information.

The ASK signal can be represented as:

$$s(t) = A_i \cos(2\pi f_c t) \tag{1}$$

where A_i is the amplitude corresponding to the binary input (high amplitude for bit 1 and lower amplitude for bit 0), and f_c is the carrier frequency.

At the receiver, demodulation is performed using correlation detection, where the received signal is multiplied with the carrier and integrated over a bit duration:

$$z = \int s(t) \cos(2\pi f_c t)\, dt \tag{2}$$

The resulting value is compared with a threshold to determine the transmitted bit sequence.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,0,1,0,1,0,1,0,1])
bp = 1e-6

samples_per_bit = 100
bit = np.repeat(x, samples_per_bit)
t1 = np.linspace(0, bp*len(x), len(bit))

A1, A2 = 10, 5
br = 1 / bp
f = br * 10

t2 = np.linspace(0, bp, 100)
m = []
```

```
for bit_val in x:
    if bit_val == 1:
        y = A1 * np.cos(2*np.pi*f*t2)
    else:
        y = A2 * np.cos(2*np.pi*f*t2)
    m.extend(y)

m = np.array(m)
t3 = np.linspace(0, bp*len(x), len(m))

mn = []
for i in range(0, len(m), len(t2)):
    segment = m[i:i+len(t2)]
    carrier = np.cos(2*np.pi*f*t2)
    z = np.trapezoid(segment * carrier, t2)

    if (2*z/bp) > 7.5:
        mn.append(1)
    else:
        mn.append(0)

mn = np.array(mn)

bit_rec = np.repeat(mn, samples_per_bit)
t4 = np.linspace(0, bp*len(mn), len(bit_rec))

plt.figure(figsize=(10,6))

plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2)
plt.title("Transmitting Information as Digital Signal")
plt.ylim([-0.5, 1.5])
plt.grid()

plt.subplot(3,1,2)
plt.plot(t3, m)
plt.title("Binary ASK Modulated Signal")
plt.grid()

plt.subplot(3,1,3)
plt.plot(t4, bit_rec, linewidth=2)
plt.title("Received Information as Digital Signal")
plt.ylim([-0.5, 1.5])
plt.grid()

plt.tight_layout()
plt.show()
```

Output

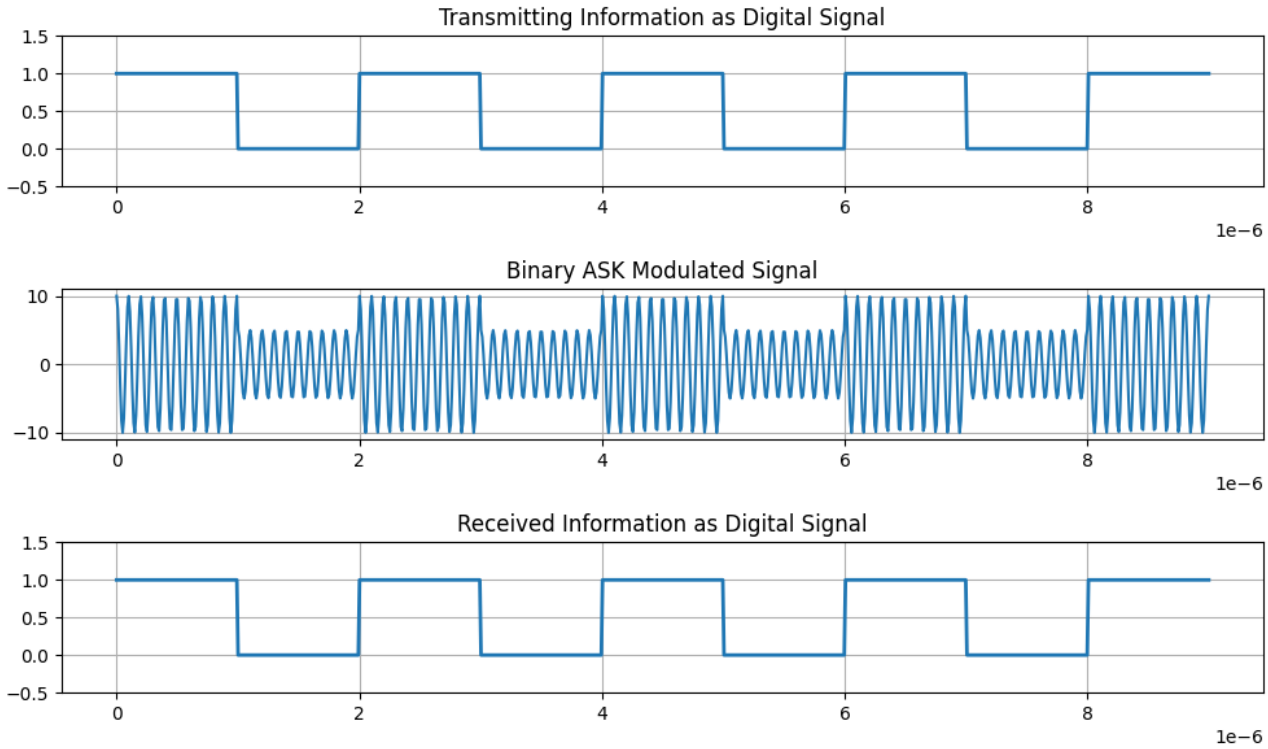

Figure 8: ASK Modulation and Demodulation Process

Results and Discussions

The ASK signal is generated by varying carrier amplitude based on input bits. Different amplitudes represent binary values in the modulated signal. Correlation detection is used to recover the transmitted data. The received signal closely matches the original input sequence. Thus, ASK modulation and demodulation are successfully implemented.

Experiment No: 9

Title

To construct a program and simulate the Phase Shift Keying (PSK) modulation technique.

Objectives

- To understand the concept of Phase Shift Keying modulation.
- To generate BPSK modulated signal from binary data.
- To recover the original signal using demodulation.

Theory

Phase Shift Keying (PSK) is a digital modulation technique in which the phase of a carrier signal is varied according to the input binary data. In Binary Phase Shift Keying (BPSK), two phases separated by π radians are used to represent binary digits.

The BPSK signal can be expressed as:

$$s(t) = A\cos(2\pi f_c t + \phi) \tag{1}$$

where $\phi = 0$ for bit 1 and $\phi = \pi$ for bit 0.

This can also be written as:

$$s(t) = \begin{cases} A\cos(2\pi f_c t), & \text{for bit 1} \\ -A\cos(2\pi f_c t), & \text{for bit 0} \end{cases} \tag{2}$$

At the receiver, demodulation is performed using correlation detection by multiplying the received signal with a reference carrier and integrating over one bit duration:

$$z = \int s(t)\cos(2\pi f_c t)\,dt \tag{3}$$

The sign of the output determines the transmitted bit.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,0,1,0,1,0,1,0,1])
bp = 1e-6

samples_per_bit = 100
bit = np.repeat(x, samples_per_bit)
t1 = np.linspace(0, bp*len(x), len(bit))

A = 5
br = 1 / bp
```

```
f = br * 2

t_bit = np.linspace(0, bp, 100)
m = []

for b in x:
    if b == 1:
        y = A * np.cos(2*np.pi*f*t_bit)
    else:
        y = -A * np.cos(2*np.pi*f*t_bit)
    m.extend(y)

m = np.array(m)
t2 = np.linspace(0, bp*len(x), len(m))

mn = []
for i in range(0, len(m), len(t_bit)):
    segment = m[i:i+len(t_bit)]
    carrier = np.cos(2*np.pi*f*t_bit)

    z = np.trapezoid(segment * carrier, t_bit)

    if (2*z/bp) > 0:
        mn.append(1)
    else:
        mn.append(0)

mn = np.array(mn)

bit_rec = np.repeat(mn, samples_per_bit)
t3 = np.linspace(0, bp*len(mn), len(bit_rec))

plt.figure(figsize=(10,6))

plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2)
plt.title("Transmitting Information as Digital Signal")
plt.ylim([-0.5, 1.5])
plt.grid()

plt.subplot(3,1,2)
plt.plot(t2, m)
plt.title("BPSK Modulated Signal")
plt.grid()

plt.subplot(3,1,3)
plt.plot(t3, bit_rec, linewidth=2)
plt.title("Received Information as Digital Signal")
plt.ylim([-0.5, 1.5])
plt.grid()

plt.tight_layout()
plt.show()
```

Output

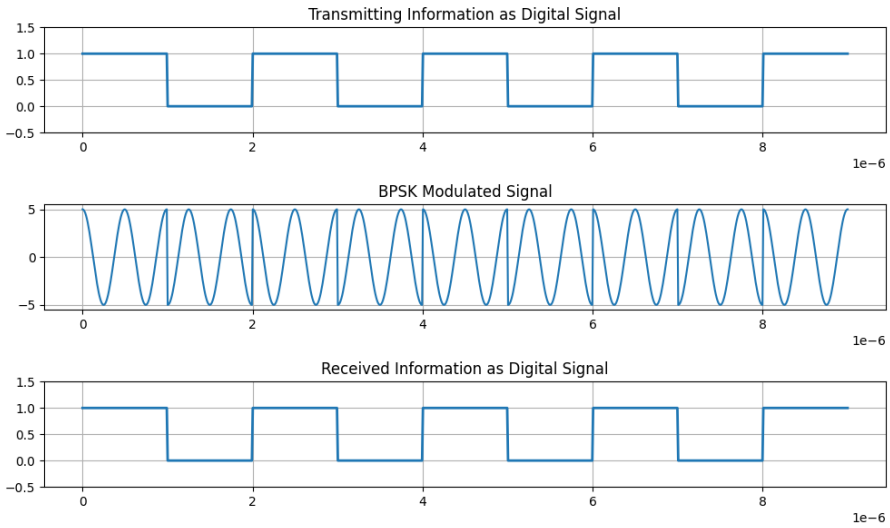

Figure 9: BPSK Modulation and Demodulation Process

Results and Discussions

The BPSK signal is generated by changing the phase of the carrier signal. Binary data is represented using phase shifts of π radians. Correlation detection is used to recover the transmitted signal. The received signal matches the original binary sequence. Thus, PSK modulation and demodulation are successfully verified.

Experiment No: 10

Title

To create a program and simulate the Frequency Shift Keying (FSK) modulation technique.

Objectives

- To understand the principle of Frequency Shift Keying modulation.
- To generate FSK modulated signal using binary data.
- To recover the original signal using demodulation.

Theory

Frequency Shift Keying (FSK) is a digital modulation technique in which the frequency of a carrier signal is varied according to the binary input data. Different frequencies represent different binary states.

The FSK signal can be expressed as:

$$s(t) = A\cos(2\pi f_i t) \tag{1}$$

where f_i represents the frequency corresponding to the input bit, and A is the amplitude of the carrier signal.

For binary FSK:

$$s(t) = \begin{cases} A\cos(2\pi f_1 t), & \text{for bit 1} \\ A\cos(2\pi f_0 t), & \text{for bit 0} \end{cases} \tag{2}$$

At the receiver, demodulation is performed using correlation detection with reference signals of both frequencies:

$$z_i = \int s(t)\cos(2\pi f_i t)\,dt \tag{3}$$

The frequency producing the higher correlation determines the transmitted bit.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 1, 0, 1, 0, 1])
bp = 1e-6

samples_per_bit = 100
bit = np.repeat(x, samples_per_bit)
t1 = np.linspace(0, bp*len(x), len(bit))

A = 5
```

```
br = 1 / bp
f1 = br * 8
f2 = br * 2

t_bit = np.linspace(0, bp, 100)
m = []

for b in x:
    if b == 1:
        y = A * np.cos(2*np.pi*f1*t_bit)
    else:
        y = A * np.cos(2*np.pi*f2*t_bit)
    m.extend(y)

m = np.array(m)
t2 = np.linspace(0, bp*len(x), len(m))

mn = []
for i in range(0, len(m), len(t_bit)):
    segment = m[i:i+len(t_bit)]

    carrier1 = np.cos(2*np.pi*f1*t_bit)
    carrier2 = np.cos(2*np.pi*f2*t_bit)

    z1 = np.trapezoid(segment * carrier1, t_bit)
    z2 = np.trapezoid(segment * carrier2, t_bit)

    if (2*z1/bp) > (A/2):
        mn.append(1)
    else:
        mn.append(0)

mn = np.array(mn)

bit_rec = np.repeat(mn, samples_per_bit)
t3 = np.linspace(0, bp*len(mn), len(bit_rec))

plt.figure(figsize=(10,6))

plt.subplot(3,1,1)
plt.plot(t1, bit, linewidth=2)
plt.title("Transmitting Information as Digital Signal")
plt.ylim([-0.5, 1.5])
plt.grid()

plt.subplot(3,1,2)
plt.plot(t2, m)
plt.title("Binary FSK Modulated Signal")
plt.grid()

plt.subplot(3,1,3)
plt.plot(t3, bit_rec, linewidth=2)
plt.title("Received Information as Digital Signal")
```

```
plt.ylim([-0.5, 1.5])
plt.grid()

plt.tight_layout()
plt.show()
```

Output

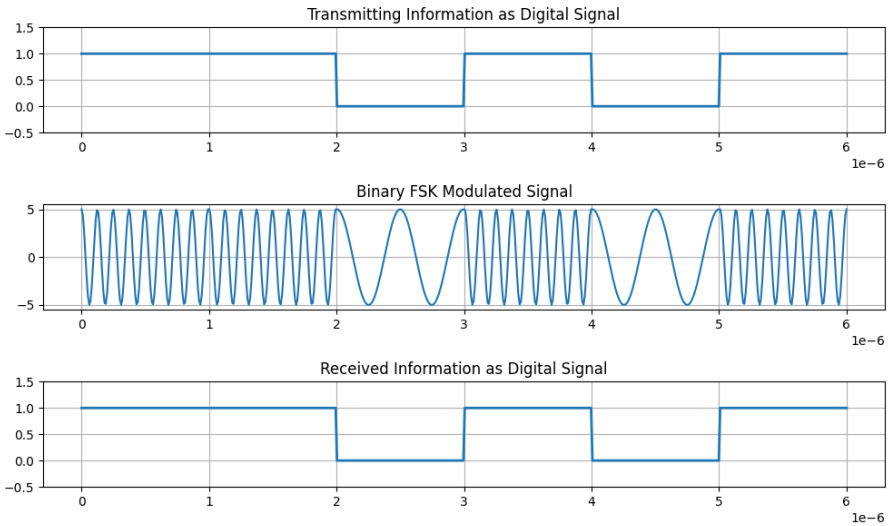

Figure 10: FSK Modulation and Demodulation Process

Results and Discussions

The FSK signal is generated by varying the frequency of the carrier. Different frequencies represent binary values in the modulated signal. Correlation detection is used to identify the transmitted frequency. The recovered signal matches the original binary sequence. Thus, FSK modulation and demodulation are successfully implemented.

Experiment No: 11

Title

To implement a program and simulate the Unipolar Non-Return-to-Zero (NRZ) signaling technique.

Objectives

- To understand the concept of Unipolar NRZ signaling.
- To generate NRZ waveform from binary data.
- To analyze the time-domain representation of digital signals.

Theory

Unipolar Non-Return-to-Zero (NRZ) is a line coding technique in which binary data is represented using a single polarity. In this scheme, a binary '1' is represented by a positive voltage level, while a binary '0' is represented by zero voltage.

The NRZ signal can be expressed as:

$$s(t) = \begin{cases} A, & \text{for bit 1} \\ 0, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the amplitude of the signal.

In NRZ signaling, the signal level remains constant throughout the bit duration and does not return to zero between consecutive bits, which improves bandwidth efficiency.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

samples_per_bit = 100
y = np.repeat(n, samples_per_bit)

t = np.linspace(0, N, len(y))

plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Unipolar NRZ Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
```

```
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

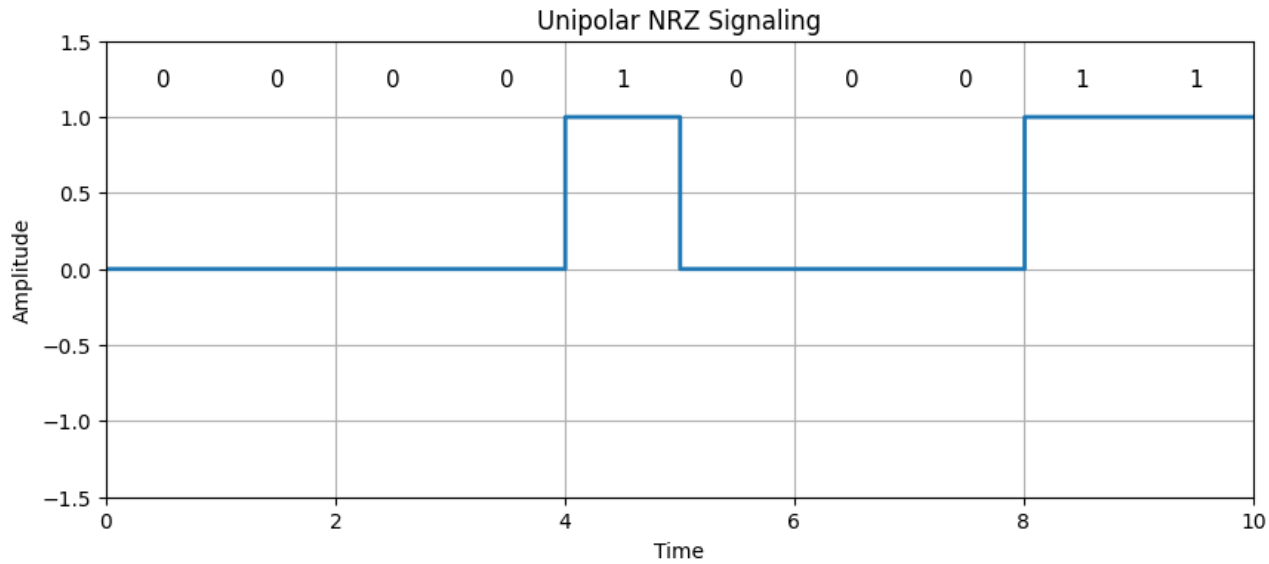

Figure 11: Unipolar NRZ Signaling Waveform

Results and Discussions

The NRZ signal is generated directly from binary input data. Binary 1 is represented by a high level and 0 by zero level. The signal remains constant during each bit interval. No return to zero occurs between consecutive bits. Thus, Unipolar NRZ signaling is successfully implemented.

Experiment No: 12

Title

To model a program and simulate the Polar Non-Return-to-Zero (NRZ) signaling technique.

Objectives

- To understand the concept of Polar NRZ signaling.
- To generate Polar NRZ waveform from binary data.
- To analyze signal representation using positive and negative levels.

Theory

Polar Non-Return-to-Zero (NRZ) is a line coding technique in which binary data is represented using two voltage levels of opposite polarity. A binary '1' is represented by a positive voltage, while a binary '0' is represented by a negative voltage. This approach improves signal balance and reduces DC components compared to unipolar signaling.

The Polar NRZ signal can be expressed as:

$$s(t) = \begin{cases} +A, & \text{for bit 1} \\ -A, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the signal amplitude. The signal level remains constant throughout the bit duration and does not return to zero between successive bits. The use of bipolar levels helps in better synchronization and improves noise immunity.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

nn = np.where(n == 1, 1, -1)

samples_per_bit = 100
y = np.repeat(nn, samples_per_bit)

t = np.linspace(0, N, len(y))

plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Polar NRZ Signaling")
```

```
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

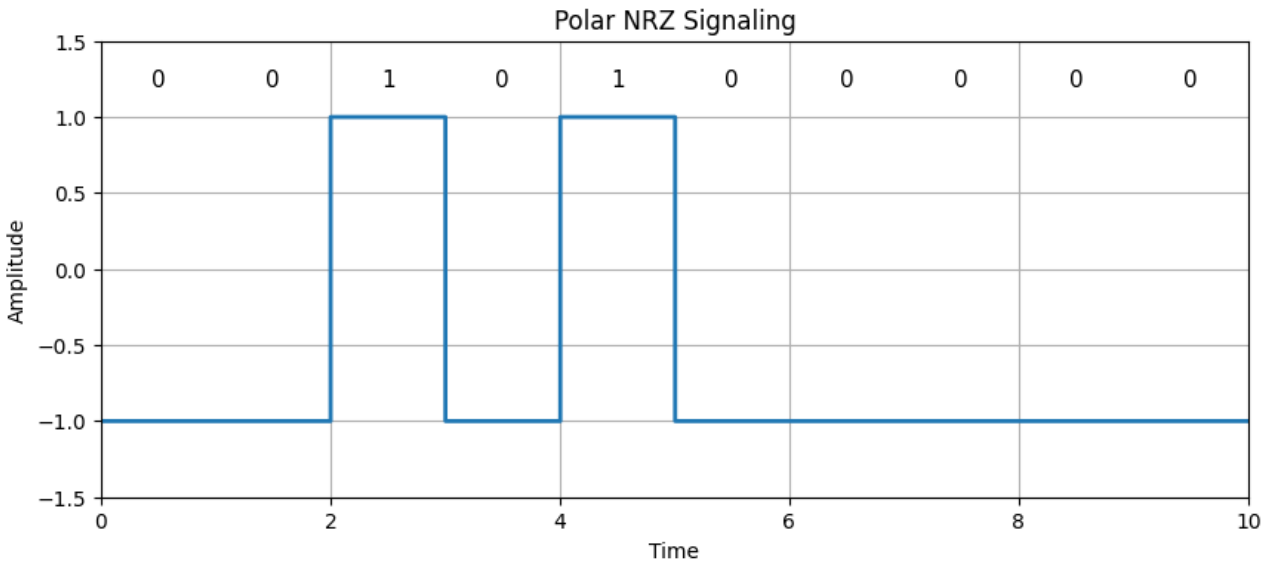

Figure 12: Polar NRZ Signaling Waveform

Results and Discussions

The generated waveform clearly shows two distinct levels for binary data. Positive amplitude represents bit 1 while negative amplitude represents bit 0. The signal remains constant over each bit duration without returning to zero. Use of bipolar levels improves signal balance and reduces DC component. Thus, Polar NRZ provides better performance compared to unipolar signaling.

Experiment No: 13

Title

To write a program and simulate the Unipolar Return-to-Zero (RZ) signaling technique.

Objectives

- To understand the principle of Unipolar RZ signaling.
- To generate RZ waveform from binary data.
- To analyze signal characteristics and transitions.

Theory

Unipolar Return-to-Zero (RZ) is a line coding technique in which binary data is represented using a single polarity, and the signal returns to zero within each bit duration. A binary '1' is represented by a positive pulse for half of the bit period followed by zero, while a binary '0' remains at zero throughout the bit duration.

The RZ signal can be expressed as:

$$s(t) = \begin{cases} A, & 0 \leq t < \frac{T}{2} \quad \text{for bit 1} \\ 0, & \frac{T}{2} \leq t < T \\ 0, & 0 \leq t < T \quad \text{for bit 0} \end{cases} \tag{1}$$

where A is the amplitude and T is the bit duration. The return-to-zero characteristic ensures better synchronization but increases bandwidth requirements compared to NRZ signaling.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10; n = np.random.randint(0, 2, N)
samples_per_bit = 100
half = samples_per_bit // 2
y = []
for bit in n:
    if bit == 1:
        y.extend([1]*half + [0]*half)
    else:
        y.extend([0]*samples_per_bit)

y = np.array(y)
t = np.linspace(0, N, len(y))
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
```

```
plt.ylim(-1.5, 1.5)
plt.title("Unipolar Return-to-Zero (RZ) Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)
plt.show()
```

Output

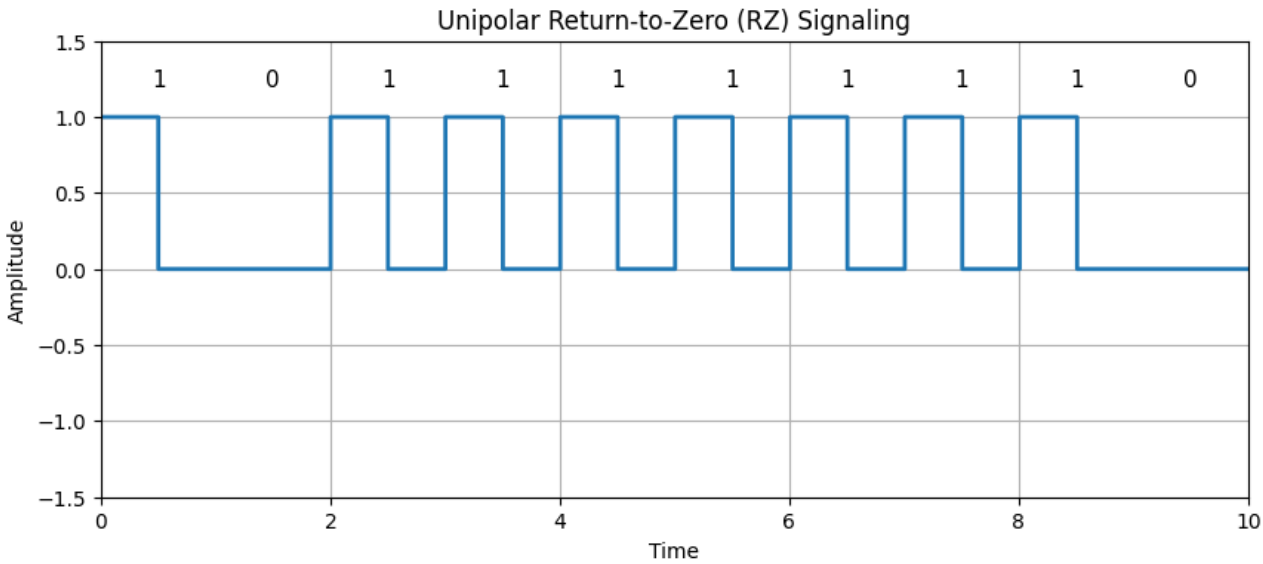

Figure 13: Unipolar Return-to-Zero (RZ) Signaling Waveform

Results and Discussions

The waveform shows pulses that return to zero within each bit interval. Binary 1 is represented by a half-duration pulse followed by zero level. Binary 0 remains at zero throughout the bit duration. Frequent transitions improve synchronization at the receiver. However, increased transitions require higher bandwidth compared to NRZ.

Experiment No: 14

Title

To simulate a program for the Bipolar Return-to-Zero (RZ) signaling technique.

Objectives

- To understand Bipolar RZ (AMI) signaling technique.
- To generate waveform with alternating polarity pulses.
- To analyze signal characteristics and DC balance.

Theory

Bipolar Return-to-Zero (RZ), also known as Alternate Mark Inversion (AMI), is a line coding technique in which binary '1's are represented by alternating positive and negative voltage pulses, while binary '0's are represented by zero voltage. In addition, the signal returns to zero within each bit duration.

The Bipolar RZ signal can be expressed as:

$$s(t) = \begin{cases} +A, & \text{for bit 1 (positive pulse)} \\ -A, & \text{for bit 1 (negative pulse)} \\ 0, & \text{for bit 0} \end{cases} \tag{1}$$

The polarity of successive '1's alternates to maintain a zero DC component. The return-to-zero property ensures that each pulse occupies only a portion of the bit duration, improving synchronization.

This technique combines the advantages of bipolar signaling and RZ format, making it suitable for reliable data transmission.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10; n = np.random.randint(0, 2, N)
nn = np.zeros(N)
polarity = 1
for i in range(N):
    if n[i] == 1:
        nn[i] = polarity
        polarity *= -1
    else:
        nn[i] = 0
samples_per_bit = 100
half = samples_per_bit // 2

y = []
```

```
for val in nn:
    if val != 0:
        y.extend([val]*half + [0]*half)
    else:
        y.extend([0]*samples_per_bit)

y = np.array(y)
t = np.linspace(0, N, len(y))
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Bipolar Return-to-Zero (BRZ) Signaling")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)
plt.show()
```

Output

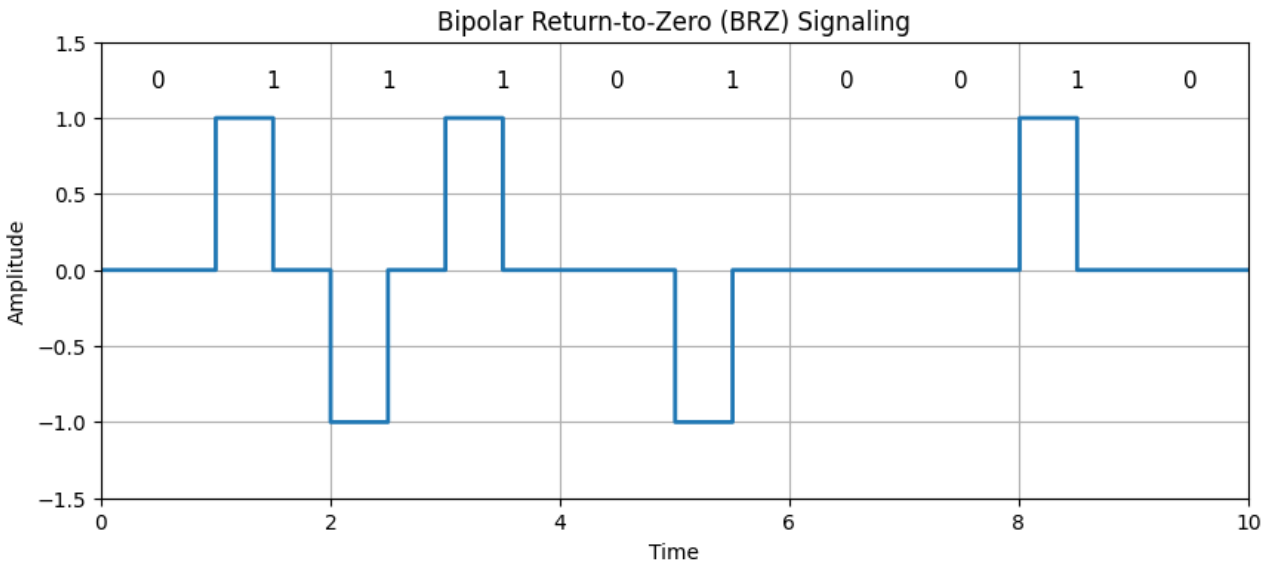

Figure 14: Bipolar Return-to-Zero (RZ) Signaling Waveform

Results and Discussions

The waveform shows alternating positive and negative pulses for binary 1. Binary 0 is represented by zero amplitude throughout the bit duration. The signal returns to zero within each bit interval. Alternating polarity ensures zero DC component and better signal balance. Thus, Bipolar RZ provides improved synchronization and transmission performance.

Experiment No: 15

Title

To develop a program and simulate the Manchester (Split-Phase) coding technique.

Objectives

- To understand the principle of Manchester encoding.
- To generate Manchester coded waveform from binary data.
- To analyze synchronization and signal transitions.

Theory

Manchester coding, also known as split-phase encoding, is a line coding technique in which each bit is represented by a transition in the middle of the bit duration. This ensures that the signal contains regular transitions, which helps in clock synchronization at the receiver.

In Manchester encoding, a binary '1' is represented by a high-to-low transition, while a binary '0' is represented by a low-to-high transition within each bit period.

The Manchester signal can be expressed as:

$$s(t) = \begin{cases} +A \to -A, & \text{for bit 1} \\ -A \to +A, & \text{for bit 0} \end{cases} \tag{1}$$

where A is the signal amplitude. Since each bit contains a transition, the average DC component is zero, and timing information is embedded within the signal itself.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

N = 10
n = np.random.randint(0, 2, N)

encoded = []

for bit in n:
    if bit == 1:
        encoded.extend([1, -1])
    else:
        encoded.extend([-1, 1])

samples_per_half_bit = 50
y = np.repeat(encoded, samples_per_half_bit)

t = np.linspace(0, N, len(y))
```

```
plt.figure(figsize=(10,4))
plt.step(t, y, where='post', linewidth=2)

plt.xlim(0, N)
plt.ylim(-1.5, 1.5)
plt.title("Manchester Coding")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()

for i in range(N):
    plt.text(i + 0.5, 1.2, str(n[i]), ha='center', fontsize=11)

plt.show()
```

Output

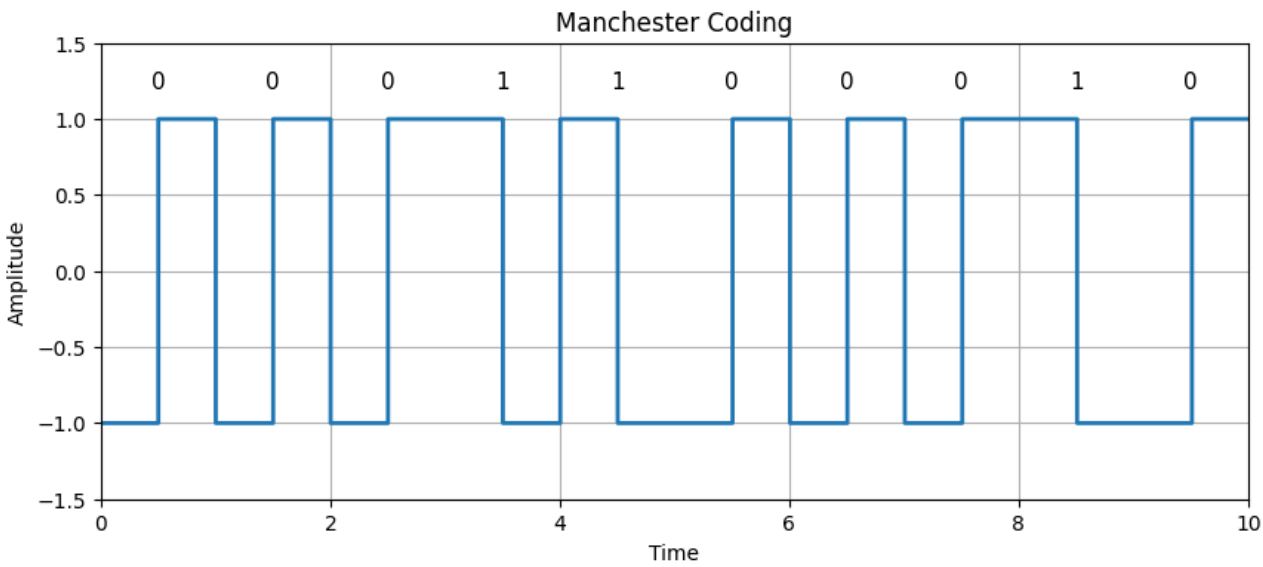

Figure 15: Manchester Coding Waveform

Results and Discussions

The waveform shows a transition in the middle of every bit period. Binary 1 produces a high-to-low transition, while 0 produces a low-to-high transition. The presence of transitions ensures reliable clock synchronization. The signal has no DC component due to equal positive and negative levels. Thus, Manchester coding provides efficient and reliable data transmission.